

项目中首页和启动页的优化建议

项目中首页和启动页的实现方式及优化建议

一、启动页实现

1. 核心实现方式

项目启动页通过 “系统原生启动屏 + 应用初始化流程” 组合实现，确保启动时的快速展示与功能准备：

(1) iOS 标准启动屏文件（LaunchScreen.storyboard）

- **作用：**负责应用启动瞬间的静态界面展示（如品牌 Logo、背景色），由系统直接渲染，无代码执行延迟，加载速度快；
- **路径：**项目中位于 `Base.lproj/LaunchScreen.storyboard`，支持可视化配置，无需编写代码即可适配不同屏幕尺寸。

(2) AppDelegate 初始化流程（核心逻辑）

应用启动完成后，在 `AppDelegate` 中完成窗口创建、根控制器设置、核心功能初始化，代码如下：

```
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(
NSDictionary *)launchOptions {
    // 1. 创建应用主窗口（全屏覆盖屏幕）
    self.window = [[UIWindow alloc] initWithFrame:[UIScreen mainScreen].bounds];
    self.window.backgroundColor = [UIColor whiteColor];

    // 2. 设置根控制器（底部标签栏控制器，管理首页、消息等页面）
    self.mainTabBar = [MainTabBarController new];
    self.window.rootViewController = self.mainTabBar;

    // 3. 初始化工具类与SDK
    [self setIQKeyboardManager]; // 键盘管理工具（自动适配输入框）
    [self setMapManager]; // 百度地图SDK（定位、地图展示功能）
}
```

```
// 4. 初始化即时通讯SDK（融云，支撑聊天功能）
[[RCIM sharedRCIM] initWithAppKey:kAppKey_RongIM];

// 5. 显示窗口（让界面可见）
[self.window makeKeyAndVisible];
return YES;
}
```

二、首页实现

首页 `ZYMainViewController` 是基于 `UITableView` 的复杂功能页面，整合“内容展示、交互、数据加载”于一体，核心实现如下：

1. 整体布局（viewDidLoad 中初始化）

```
- (void)viewDidLoad {
    [super viewDidLoad];
    self.title = @"首页"; // 页面标题

    // 1. 配置导航栏（含搜索框、功能按钮）
    [self setupNavItems];

    // 2. 设置表格头视图（轮播广告图，提升首页吸引力）
    self.tableView.tableHeaderView = self.adView;

    // 3. 初始化数据数组（存储不同类型内容，如新房、二手房）
    listShowdatas = [[NSMutableArray alloc] init]; // 基础展示数据
    listHotNewData = [[NSMutableArray alloc] init]; // 热门资讯数据
    // ...其他业务数据数组初始化（根据内容类型拆分）

    // 4. 配置下拉刷新（使用MJRefresh框架，获取最新数据）
    __weak typeof(self) weakSelf = self; // 弱引用，避免循环引用
    self.tableView.mj_header = [MJRefreshNormalHeader headerWithRefreshingBlock:^(
        [weakSelf listCityUrl]; // 触发数据刷新（核心请求方法）
    )];
}
```

2. 导航栏自定义（中间搜索框）

首页导航栏未使用常规标题，而是替换为搜索框，提升用户查找效率，代码如下：

```
- (void)setupNavItems {  
    // 1. 导航栏样式设置（背景色、阴影等）  
    self.navigationController.navigationBar.backgroundColor = [UIColor whiteColor];  
  
    // 2. 创建搜索框视图（自定义视图 ZYMainSerachView）  
    self.searchView = [[ZYMainSerachView alloc] initWithFrame:CGRectMake(0, 0, SCREEN_WIDTH - 30, 30)];  
    self.searchView.frame = CGRectMake(0, 0, SCREEN_WIDTH - 30, 30); // 宽度适配屏幕（左右留空15px）  
    self.navigationItem.titleView = self.searchView; // 将搜索框设为导航栏标题视图  
  
    // 3. 设置搜索框代理（处理搜索点击、输入回调）  
    self.searchView.delegate = self;  
}
```

3. 数据加载机制（自动刷新）

页面即将显示时，自动触发下拉刷新，确保用户看到最新内容，无需手动操作：

```
- (void)viewWillAppear:(BOOL)animated {  
    [super viewWillAppear:animated];  
    // 自动触发下拉刷新（进入页面即加载最新数据）  
    [self.tableView.mj_header beginRefreshing];  
}
```

4. 内容展示结构（多分区组织）

首页通过 UITableView 的多分区（section）组织不同类型内容，每个分区使用自定义 UITableViewCell 适配展示，结构清晰：

- **分区 1：** 轮播广告（表格头视图 advView）；
- **分区 2：** 新房推荐（使用 NewHouseCell，展示房价、户型）；
- **分区 3：** 热门活动（使用 ActivityCell，展示活动图片、时间）；

- **分区 4**：二手房列表（使用 `SecondHouseCell`，展示面积、朝向）；
- **分区 5**：租房信息（使用 `RentHouseCell`，展示租金、位置）；
- ...（其他业务分区，根据产品需求扩展）。

三、优化建议

1. 启动页优化（提升启动体验）

（1）添加启动动画（增强品牌感知）

通过简单动画过渡启动屏与首页，避免界面切换生硬，代码如下：

```
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(
NSDictionary *)launchOptions {
    // 现有初始化代码（窗口、根控制器、SDK初始化） ...

    // 1. 创建启动动画视图（覆盖全屏，置于最上层）
    UIView *launchAnimationView = [[UIView alloc] initWithFrame:self.window.bounds];
    launchAnimationView.backgroundColor = [UIColor whiteColor]; // 背景色与启动屏一致，过渡
    自然

    // 2. 添加品牌Logo（居中显示，强化品牌认知）
    UIImageView *logoView = [[UIImageView alloc] initWithImage:[UIImage imageNamed:@"app
_logo"]];
    logoView.center = launchAnimationView.center; // 垂直+水平居中
    logoView.bounds = CGRectMake(0, 0, 80, 80); // Logo尺寸（根据设计调整）
    [launchAnimationView addSubview:logoView];

    // 3. 显示动画视图
    [self.window addSubview:launchAnimationView];
    [self.window bringSubviewToFront:launchAnimationView];

    // 4. 执行淡入淡出动画（1.5秒后移除动画视图，过渡到首页）
    [UIView animateWithDuration:1.0 animations:^(
        // 可选：添加Logo缩放动画（从0.8→1.0，增强动感）
        logoView.transform = CGAffineTransformMakeScale(1.0, 1.0);
    ) completion:^(BOOL finished) {
```

```

[UIView animateWithDuration:0.5 animations:^(
    launchAnimationView.alpha = 0; // 渐隐动画
) completion:^(BOOL finished) {
    [launchAnimationView removeFromSuperview]; // 动画结束后移除视图，释放内存
}];

return YES;
}

```

(2) 启动数据预加载（提升首页加载速度）

在启动页阶段异步预加载基础数据（如城市列表、配置信息），避免首页加载时重复请求，代码如下：

```

- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(
NSDictionary *)launchOptions {
    // 现有初始化代码...

    // 异步预加载基础数据（使用全局队列，不阻塞主线程）
    dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0), ^{
        [self preloadBaseData]; // 预加载核心基础数据

        // 数据加载完成后，通知首页（可选，避免首页重复加载）
        dispatch_async(dispatch_get_main_queue(), ^{
            [[NSNotificationCenter defaultCenter] postNotificationName:@"BaseDataLoaded" object
:nil];
        });
    });

    return YES;
}

// 预加载具体实现（基础数据，非首页即时展示但必要）
- (void)preloadBaseData {
    // 1. 预加载城市列表（缓存到本地，首页直接使用）
    [CityDataManager preloadCityListFromLocalOrRemote];

    // 2. 预加载应用配置（如首页轮播图地址、功能开关）
    [ConfigManager loadRemoteConfigWithCompletion:nil];
}

```

```
// 3. 预加载用户基础信息（如登录状态、用户ID，避免首页重复判断）
[UserManager preloadUserInfo];
}
```

2. 首页优化（提升加载速度与稳定性）

（1）延迟加载非关键资源（优先展示首屏）

将非首屏核心数据（如底部资讯、历史记录）延迟加载，减少首屏渲染压力，代码如下：

```
- (void)viewDidLoad {
    [super viewDidLoad];

    // 1. 优先加载首屏核心数据（轮播图、置顶推荐、新房列表）
    [self loadCoreData];

    // 2. 延迟1秒加载非核心数据（不影响首屏展示，提升加载速度）
    dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(1.0 * NSEC_PER_SEC)),
        dispatch_get_main_queue(), ^{
            [self loadSecondaryData]; // 非核心数据：热门资讯、租房列表、浏览历史
        });
}

// 加载首屏核心数据（用户进入首页第一眼看到的内容）
- (void)loadCoreData {
    [self loadBannerData]; // 轮播广告图
    [self loadTopRecommend]; // 置顶推荐（新房/二手房）
    [self loadNewHouseList]; // 新房列表（首屏主要内容）
}

// 加载非核心数据（延迟加载，不影响首屏渲染）
- (void)loadSecondaryData {
    [self loadHotNews]; // 热门资讯（底部区域）
    [self loadRentHouseList]; // 租房列表（非首屏重点）
    [self loadHistoryRecords]; // 用户浏览历史（个性化内容）
}
```

（2）内存优化（避免泄漏与冗余）

页面销毁时，及时清理代理、数据数组，释放内存，避免野指针与内存泄漏：

```
- (void)dealloc {
    // 1. 清理代理（避免循环引用，如表格、搜索框）
    self.tableView.delegate = nil;
    self.tableView.dataSource = nil;
    self.searchView.delegate = nil;

    // 2. 移除通知监听（避免重复回调，如数据加载完成通知）
    [[NSNotificationCenter defaultCenter] removeObserver:self];

    // 3. 清空数据数组（释放内存，避免冗余数据占用）
    [self clearAllDataArrays];
}
// 清空所有业务数据数组
- (void)clearAllDataArrays {
    listShowdatas = nil;
    listHotNewData = nil;
    // ...其他业务数据数组置空（按初始化顺序对应）
}
```

(3) 拆分复杂逻辑（减少 Controller 代码量）

将首页的“数据请求、业务处理”逻辑拆分到独立类（如 `DataManager`），让 Controller 仅负责 UI 展示与交互，提升代码可维护性：

```
// 1. 定义数据管理类（ZYMainDataManager.h）
@interface ZYMainDataManager : NSObject
// 加载首页核心数据
- (void)loadHomeCoreDataWithCompletion:(void (^)(BOOL success, NSDictionary *data))
completion;
// 加载首页非核心数据
- (void)loadHomeSecondaryDataWithCompletion:(void (^)(BOOL success, NSDictionary *data
))completion;
@end
// 2. 数据管理类实现（ZYMainDataManager.m）
@implementation ZYMainDataManager
- (void)loadHomeCoreDataWithCompletion:(void (^)(BOOL success, NSDictionary *))
completion {
```

```

// 1. 发送网络请求（首页核心数据：轮播图、新房列表）
NSDictionary *parameters = @{@"cityId": [CityDataManager currentCityId]};
[ZYNetworkRequest postBeanNoPrompt:parameters url:kHomeCoreDataURL success:^(id
json) {
    ZYNetWorkModel *model = (ZYNetWorkModel *)json;
    if ([model.status isEqualToString:@"1"]) {
        NSDictionary *data = @{
            @"banner": model.data[@"banner"],
            @"newHouse": model.data[@"newHouse"]
        };
        completion(YES, data); // 回调成功数据
    } else {
        completion(NO, nil); // 回调失败
    }
} failure:^(NSError *error) {
    completion(NO, nil); // 网络错误回调
} progress:^(float progress) {}];
}
@end

// 3. 首页 Controller 调用（仅负责 UI 与交互）
@implementation ZYMainViewController
- (void)viewDidLoad {
    [super viewDidLoad];
    // 初始化数据管理类
    self.dataManager = [[ZYMainDataManager alloc] init];
    // 配置 UI（导航栏、表格等）
    [self setupUI];
    // 加载数据（通过数据管理类，不直接写请求逻辑）
    [self loadDataByManager];
}
- (void)loadDataByManager {
    // 加载核心数据
    [self.dataManager loadHomeCoreDataWithCompletion:^(BOOL success, NSDictionary *data
) {
        if (success) {
            // 更新 UI（轮播图、新房列表）
            self.adView.bannerData = data[@"banner"];
            listShowdatas = data[@"newHouse"];
            [self.tableView reloadData];
        }
    }];
}

```



```
}  
@end
```

(4) 优化下拉刷新体验（提升用户感知）

添加刷新状态提示、失败重试机制，让用户清晰了解加载进度，代码如下：

```
- (void)viewDidLoad {  
    [super viewDidLoad];  
    // ...其他初始化代码  
  
    // 1. 配置下拉刷新回调（添加状态提示）  
    __weak typeof(self) weakSelf = self;  
    self.tableView.mj_header.refreshingBlock = ^{  
        [weakSelf showRefreshStatus:@"正在获取最新数据..."]; // 显示加载提示  
        [weakSelf listCityUrl]; // 触发数据请求  
    };  
  
    // 2. 配置刷新组件样式（提升视觉体验）  
    MJRefreshNormalHeader *header = (MJRefreshNormalHeader *)self.tableView.mj_header;  
    header.stateLabel.font = [UIFont systemFontOfSize:12]; // 状态文字字体  
    [header setTitle:@"下拉可刷新" forState:MJRefreshStateIdle];  
    [header setTitle:@"松开立即刷新" forState:MJRefreshStatePulling];  
    [header setTitle:@"正在刷新..." forState:MJRefreshStateRefreshing];  
}  
// 显示刷新状态提示（如顶部提示条）  
- (void)showRefreshStatus:(NSString *)status {  
    self.refreshStatusLabel.text = status;  
    self.refreshStatusLabel.hidden = NO;  
}  
// 数据请求失败处理（提供重试按钮）  
- (void)requestFailed {  
    // 1. 结束刷新动画  
    if (self.tableView.mj_header.isRefreshing) {  
        [self.tableView.mj_header endRefreshing];  
    }  
  
    // 2. 显示错误提示与重试按钮  
    [self showErrorViewWithMessage:@"数据加载失败" retryAction:^(  
        // 重试回调：重新触发刷新
```

```
[self.tableView.mj_header beginRefreshing];  
    }  
}
```

总结

启动页优化核心是“提升过渡体验 + 预加载基础数据”，减少首页等待时间；首页优化核心是“优先首屏渲染 + 拆分复杂逻辑 + 内存清理”，提升加载速度与代码可维护性。两者结合可显著改善应用的启动效率与首页用户体验，符合 iOS 应用性能优化的核心目标。

（注：文档部分内容可能由 AI 生成）